数据库管理系统原理与实现

杜小勇 陈红 卢卫 主编

深圳技术大学

大数据与互联网学院

第3章 关系数据库存储管理

- 3.1 SQL概述
- 3.2 数据定义
- 3.3 基本SQL查询
- 3.4 数据更新(先讲)
- 3.5 视图
- 3.6 小结

3.4 数据更新

- 3.4.1 插入数据
- 3.4.2 修改数据
- 3.4.3 删除数据

3.5.1 SQL插入数据

- 两种SQL插入数据方式
  - 插入元组
  - 插入子查询结果
    - 可以一次插入多个元组

1. 插入元组

- 语句格式

  INSERT

  INTO <表名> [(<属性列1>[,<属性列2 >…)]

  VALUES (<常量1> [,<常量2>]…);

- 功能
  - 将新元组插入指定表中

插入元组（续）

- INTO子句
  - 指定要插入数据的表名及属性列
  - 属性列的顺序可与表定义中的顺序不一致
  - 没有指定属性列：表示要插入的是一条完整的元组，且属性列属性与表定义中的顺序一致
  - 指定部分属性列：插入的元组在其余属性列上取空值

❖**VALUES子句**

■ **提供的值必须与INTO子句匹配**

● **值的个数、类型**

插入多行数据

- 语句格式

INSERT

INTO <表名> [(<属性列1>[,<属性列2 >…)]

VALUES (<常量1> [,<常量2>]…), #逗号分隔多行

(<常量3> [,<常量4>]…),

… (<常量n> [,<常量m>]…);

- 功能
  - 将新元组插入指定表中

3.4.1 插入数据(教材)

插入语句以INSERT INTO 开始，后面是表的名称。例如：

INSERT INTO TotalPrice (OID, TotalPrice) 未完

<insert statement> ::=

INSERT INTO <table name> <insert columns and source>

<insert columns and source>：即插入的属性列， source则是查询表达式或是由VALUES引出的一组插入的值（即一个元组）。

<insert columns and source> ::=

[<left paren> <insert column list> <right paren>] <query expression>

| DEFAULT VALUES

<insert column list> ::= <column name list>

3.4.1 插入数据

[例3.19]在供应商表中新增一条记录，其供应商ID为S004，名称为宝塔山印刷厂，城市为延安。

INSERT INTO Suppliers VALUES('S004','宝塔山印刷厂','延安'); /*这样的缺点是什么？ */

由于省略了<insert column list>，后面VALUES包含的元组必须包含Suppliers的全部属性。

```
CREATE TABLE Suppliers(
        SID VARCHAR(32) PRIMARY KEY,    /*供应商ID（主码）*/
        SName VARCHAR(128) NOT NULL,    /*供应商名称*/
        City VARCHAR(128)               /*供应商所在城市*/
);
```

#mysql可以给非字符串数据加单引号或双引号，但int型可以不加

Mysql语法

INSERT [LOW_PRIORITY | DELAYED | HIGH_PRIORITY] [IGNORE] [INTO] *tbl_name* [PARTITION (*partition_name* [, *partition_name*] ...)] [(*col_name* [, *col_name*] ...)] { {VALUES | VALUE} (*value_list*) [, (*value_list*)] ... } [AS *row_alias*[(*col_alias* [, *col_alias*] ...)]] [ON DUPLICATE KEY UPDATE *assignment_list*]

PRIORITY：仅影响表级锁
DELAYED：已无意义
IGNORE：如主码数据已存在，则不插入。

value：用于插入一行数据

ON DUPLICATE KEY UPDATE：
如主码数据已存在，则更新。

Mysql语法

```
INSERT [LOW_PRIORITY |
DELAYED | HIGH_PRIORITY]
[IGNORE] [INTO] tbl_name
[PARTITION (partition_name [,
partition_name] ...)] SET
assignment_list [AS
row_alias[(col_alias [,
col_alias] ...)]] [ON DUPLICATE
KEY UPDATE assignment_list]
```

用于插入一条记录

```
INSERT
    INTO SC
    SET sno=
‘ 201215128 ’,cno=‘ 1 ’,g
rade=NULL;
```

Mysql语法

```
INSERT [LOW_PRIORITY |
HIGH_PRIORITY] [IGNORE]
[INTO] tbl_name [PARTITION
(partition_name [,
partition_name] ...)]
[(col_name [, col_name] ...)]
{ SELECT ... | TABLE
table_name | VALUES
row_constructor_list } [ON
DUPLICATE KEY UPDATE
assignment_list]
```

用于插入查询结果

```
INSERT INTO tbl_temp2 (fld_id)
   SELECT tbl_temp1.fld_order_id
   FROM tbl_temp1
   WHERE tbl_temp1.fld_order_id
> 100;
```

Mysql语法

```
REPLACE [LOW_PRIORITY |
DELAYED]
    [INTO] tbl_name
    [PARTITION (partition_name [,
partition_name] ...)]
    [(col_name [, col_name] ...)]
    { {VALUES | VALUE} (value_list) [,
(value_list)] ...
       |
      VALUES row_constructor_list
    }
```

插入数据时，如果数据存在，则删除再插入

插入和修改数据（续）

- 关系数据库管理系统在执行插入/修改语句时会检查所插元组是否破坏表上已定义的完整性规则
  - 实体完整性
  - 参照完整性
  - 用户定义的完整性
    - NOT NULL约束
    - UNIQUE约束
    - 值域约束

3.4 数据更新

- 3.4.1 插入数据
- 3.4.2 修改数据
- 3.4.3 删除数据

3.4.2 修改数据

- 语句格式

  UPDATE <表名>
  SET <列名>=<表达式>[,<列名>=<表达式>]…
  [WHERE <条件>];

- 功能
  - 修改指定表中满足WHERE子句条件的元组
  - SET子句给出<表达式>的值用于取代相应的属性列
  - 如果省略WHERE子句，表示要修改表中的所有元组

修改数据（续）

- 三种修改方式
  - 修改某一个元组的值
  - 修改多个元组的值
  - 带子查询的修改语句

3.4.2 修改数据

[例3.21]由于数码产品降价，将所有类别为数码产品的商品价格调整为原价的90%。

```
UPDATE Products
    SET Price = Price * 0.9
    WHERE Category = '数码产品';
```

```
CREATE TABLE Products(
    PID VARCHAR(32) PRIMARY KEY,        /*商品ID （主码） */
    PName VARCHAR(128) NOT NULL,        /*商品名称*/
    Price DECIMAL,                      /*商品单价，类型为带小数的数字*/
    Category VARCHAR(128),              /*商品类别*/
    SID VARCHAR(32) NOT NULL,           /*供应商ID*/
    FOREIGN KEY (SID) REFERENCES Suppliers(SID)
);
```

<update statement: searched> ::=

UPDATE <table name> SET <set clause list> [WHERE <search condition>]

带子查询的修改语句

[例,第6版] 将计算机科学系全体学生的成绩置零。

```
UPDATE SC
SET    Grade=0
WHERE Sno  IN
      (SELECT Sno
       FROM     Student
       WHERE  Sdept= 'CS' );
```

/*但MySQL不允许对一张表查询和删除，但可以用派生表方式*/

Mysql语法

UPDATE [LOW_PRIORITY] [IGNORE] *table_reference* SET *assignment_list* [WHERE *where_condition*] [ORDER BY ...] [LIMIT *row_count*]

ORDER BY 子句用来指定数据行的更新顺序，一般不写就是按照记录聚簇索引的顺序。

LIMIT 子句限制数据更新的行数。

3.4 数据更新

- 3.4.1 插入数据
- 3.4.2 修改数据
- 3.4.3 删除数据

3.4.3 删除数据

- 语句格式

  DELETE

  FROM　　<表名>

  [WHERE <条件>];

- 功能
  - 删除指定表中满足WHERE子句条件的元组
- WHERE子句
  - 指定要删除的元组
  - 缺省表示要删除表中的全部元组，表的定义仍在数据库中

删除数据（续）

- 三种删除方式
  - 删除某一个元组的值
  - 删除多个元组的值
  - 带子查询的删除语句

3.4.3 删除数据

[例3.22]从OrderItems表中删除所有OID为'O001'的数据。

```
DELETE FROM OrderItems WHERE OID = 'O001';
```

```
DELETE FROM OrderItems;  #这样会如何？
```

<delete statement: searched> ::=

DELETE FROM <table name> [WHERE <search condition>]

带子查询的删除语句

[例,第6版] 删除计算机科学系所有学生的选课记录。

```
DELETE FROM SC
WHERE Sno IN (
    SELECT Sno FROM Student
    WHERE Sdept= 'CS');
```

/*但MySQL不允许对一张表查询和删除，但可以用派生表方式*/

Mysql语法

DELETE [LOW_PRIORITY] [QUICK] [IGNORE] FROM *tbl_name* [[AS] *tbl_alias*] [PARTITION (*partition_name* [, *partition_name*] ...)] [WHERE *where_condition*] [ORDER BY ...] [LIMIT *row_count*]

QUICK：针对MyISAM引擎提速

IGNORE：忽略过程错误

ORDER BY：子句用来指定数据行的删除顺序。

LIMIT：子句限制删除数据的行数。

Mysql语法

DELETE [LOW_PRIORITY]
[QUICK] [IGNORE]
tbl_name[.*] [,
tbl_name[.*]] ... FROM
table_references
[WHERE
where_condition]

支持从多表删除数据

Mysql高效清除表数据

TRUNCATE [TABLE] *tbl_name*

保留表结构，只高效删除该表数据

与drop table 删表有区别;

3.3 基本SQL查询

- 3.3.1 单表查询
- 3.3.2 连接查询
- 3.3.3 集合查询
- 3.3.4 空值查询
- 3.3.5 聚集查询

3.3.2 连接查询(第6版)

- 连接查询：同时涉及两个以上的表的查询
- 连接条件或连接谓词：用来连接两个表的条件

  一般格式：

  - [<表名1>.]<列名1> <比较运算符> [<表名2>.]<列名2>
  - [<表名1>.]<列名1> BETWEEN [<表名2>.]<列名2> AND [<表名2>.]<列名3>
- 连接字段：连接谓词中的列名称
  - 连接条件中的各连接字段类型必须是可比的，但名字不必相同

连接查询（续）

1.等值与非等值连接查询

2.自身连接

3.外连接

4.多表连接

1. 等值与非等值连接查询

- 等值连接：连接运算符为=
- 非等值连接：连接运算符为其他

[例,第6版] 查询每个学生及其选修课程的情况

```
SELECT  Student.*, SC.*
FROM    Student, SC
WHERE   Student.Sno = SC.Sno;
```

等值与非等值连接查询（续）

查询结果：

Student.Sno	Sname	Ssex	Sage	Sdept	SC.Sno	Cno	Grade
201215121	李勇	男	20	CS	201215121	1	92
201215121	李勇	男	20	CS	201215121	2	85
201215121	李勇	男	20	CS	201215121	3	88
201215122	刘晨	女	19	CS	201215122	2	90
201215122	刘晨	女	19	CS	201215122	3	80

等值与非等值连接查询（续）

- 自然连接—等值连接里去掉目标列重复的属性列

[例] 用自然连接完成。

```
SELECT  Student.Sno,Sname,Ssex,Sage,Sdept,Cno,Grade
FROM    Student,SC
WHERE  Student.Sno = SC.Sno;
```

❖自然连接（natural join）

自然连接(Natural join)是一种特殊的等值连接，它要求两个关系中进行比较的分量必须是相同的属性组，并且在结果中把重复的属性列去掉。而等值连接并不去掉重复的属性列，需要在select里输入列名。

如：A中a,b,c字段，B中有c,d字段

则select * from A natural join B
相当于
select A.a,A.b,A.c,B.d from A.c = B.c

连接查询（续）

等值连接与自然连接的区别为：自然连接会去重复列，当然，一般在select里指定列：

例如：

```
SELECT * FROM S,J WHERE S.CITY = J.CITY
```

正在显示第 0 - 5 行（共 6 行，查询花费 0.0005 秒。）

SNO	SNAME	STATUS	CITY	JNO	JNAME	CITY
S2	盛锡	10	北京	J1	三建	北京
S3	东方红	30	北京	J1	三建	北京
S1	精益	20	天津	J3	弹簧厂	天津
S4	丰泰盛	20	天津	J3	弹簧厂	天津
S1	精益	20	天津	J4	造船厂	天津
S4	丰泰盛	20	天津	J4	造船厂	天津

等值VS自然

```
SELECT * FROM S NATURAL JOIN J
```

正在显示第 0 - 5 行（共 6 行，查询花费 0.0003 秒。）

CITY	SNO	SNAME	STATUS	JNO	JNAME
北京	S2	盛锡	10	J1	三建
北京	S3	东方红	30	J1	三建
天津	S1	精益	20	J3	弹簧厂
天津	S4	丰泰盛	20	J3	弹簧厂
天津	S1	精益	20	J4	造船厂
天津	S4	丰泰盛	20	J4	造船厂

等值与非等值连接查询（续）

- 一条SQL语句可以同时完成选择和连接查询，这时WHERE子句是由连接谓词和选择谓词组成的复合条件。

[例,第6版]查询选修2号课程且成绩在90分以上的所有学生的学号和姓名。

```
SELECT Student.Sno, Sname
FROM    Student, SC
WHERE  Student.Sno=SC.Sno  AND
            SC.Cno=' 2 ' AND SC.Grade>90;
```

- 可能的执行过程:
  - 先从SC中挑选出Cno='2'并且Grade>90的元组形成一个中间关系
  - 再和Student中满足连接条件的元组进行连接得到最终的结果关系

连接查询（续）

1.等值与非等值连接查询

2.自身连接

3.外连接

4.多表连接

2. 自身连接

- 自身连接：一个表与其自己进行连接
- 需要给表起别名以示区别
- 由于所有属性名都是同名属性，因此必须使用别名前缀

[例 ,第6版]查询每一门课的间接先修课（即先修课的先修课，不考虑再之前的先修课，即只有一门）

```
SELECT  FIRST.Cno, SECOND.Cpno
FROM  Course  FIRST, Course  SECOND
WHERE FIRST.Cpno = SECOND.Cno and SECOND.Cpno IS NOT NULL;
```

/*间接先修课null的意义，取决于题目要求，不需要间接先修课其实也有某种意义。*/

自身连接（续）

FIRST表（Course表）

课程号 Cno	课程名 Cname	先行课 Cpno	学分 Ccredit
1	数据库	5	4
2	数学		2
3	信息系统	1	4
4	操作系统	6	3
5	数据结构	7	4
6	数据处理		2
7	PASCAL 语言	6	4

SECOND表（Course表）

课程号 Cno	课程名 Cname	先行课 Cpno	学分 Ccredit
1	数据库	5	4
2	数学		2
3	信息系统	1	4
4	操作系统	6	3
5	数据结构	7	4
6	数据处理		2
7	PASCAL 语言	6	4

查询结果：

Cno	Cpno
1	7
3	5
5	6

连接查询（续）

1.等值与非等值连接查询

2.自身连接

3.外连接

4.多表连接

3. 外连接

- 外连接与普通连接的区别
  - 普通连接操作只输出满足连接条件的元组
  - 外连接操作以指定表为连接主体，将主体表中不满足连接条件的元组一并输出
  - 左外连接
    - 列出左边关系中所有的元组
  - 右外连接
    - 列出右边关系中所有的元组

外连接（续）

[例,第6版] 查询每个学生及其选修课程的情况

```
SELECT  Student.Sno,Sname,Ssex,Sage,Sdept,Cno,Grade
FROM    Student,SC
WHERE  Student.Sno = SC.Sno;
```

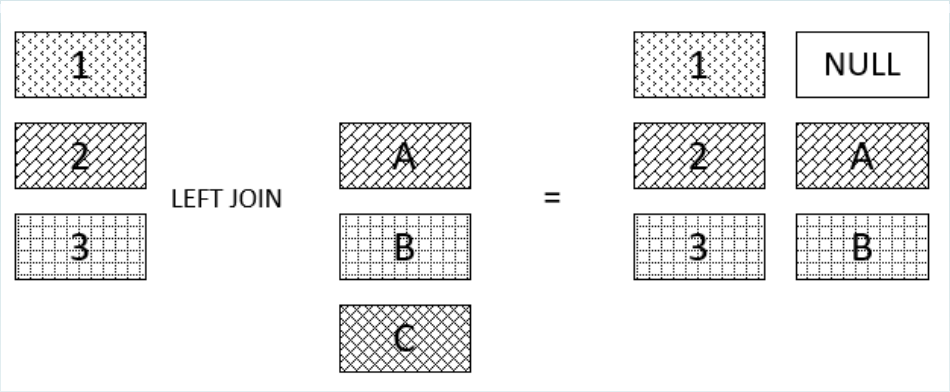
正确？ 这只是选修了课程的学生，应该为左外连接，包含没有选课的学生，也是最常用的外连接

改写:

```
SELECT Student.Sno,Sname,Ssex,Sage,Sdept,Cno,Grade
FROM  Student  LEFT JOIN SC ON    -- mysql可省略outer;
          (Student.Sno=SC.Sno);  -- 少用using(Sno)，不灵活，要求列名需一样
```

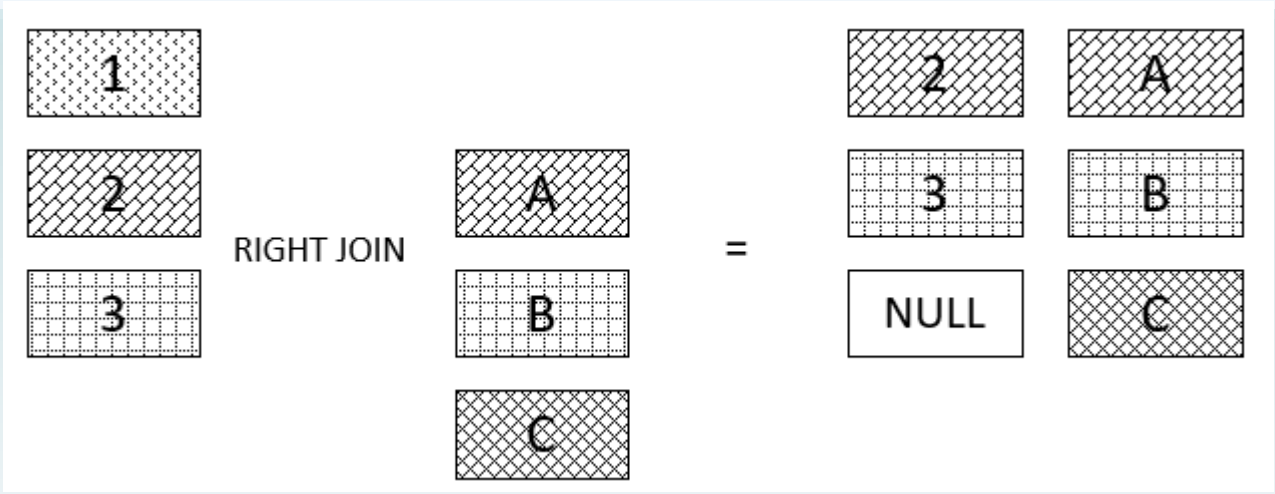
外连接（续）

执行结果：

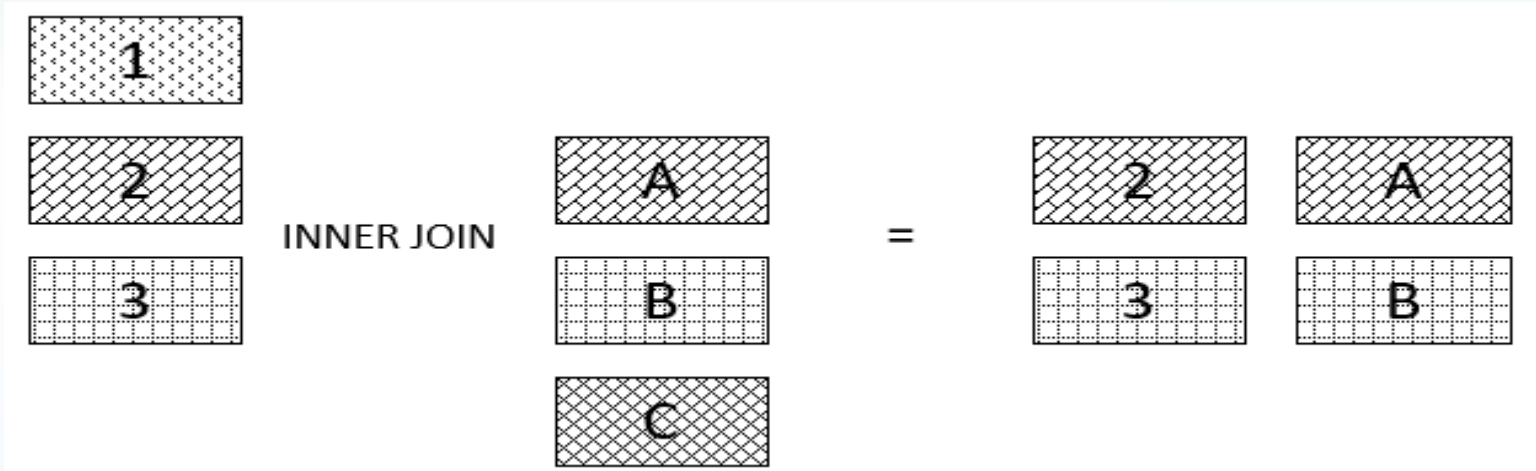


Student.Sno	Sname	Ssex	Sage	Sdept	Cno	Grade
201215121	李勇	男	20	CS	1	92
201215121	李勇	男	20	CS	2	85
201215121	李勇	男	20	CS	3	88
201215122	刘晨	女	19	CS	2	90
201215122	刘晨	女	19	CS	3	80
201215123	王敏	女	18	MA	NULL	NULL
201215125	张立	男	19	IS	NULL	NULL

外连接（续）

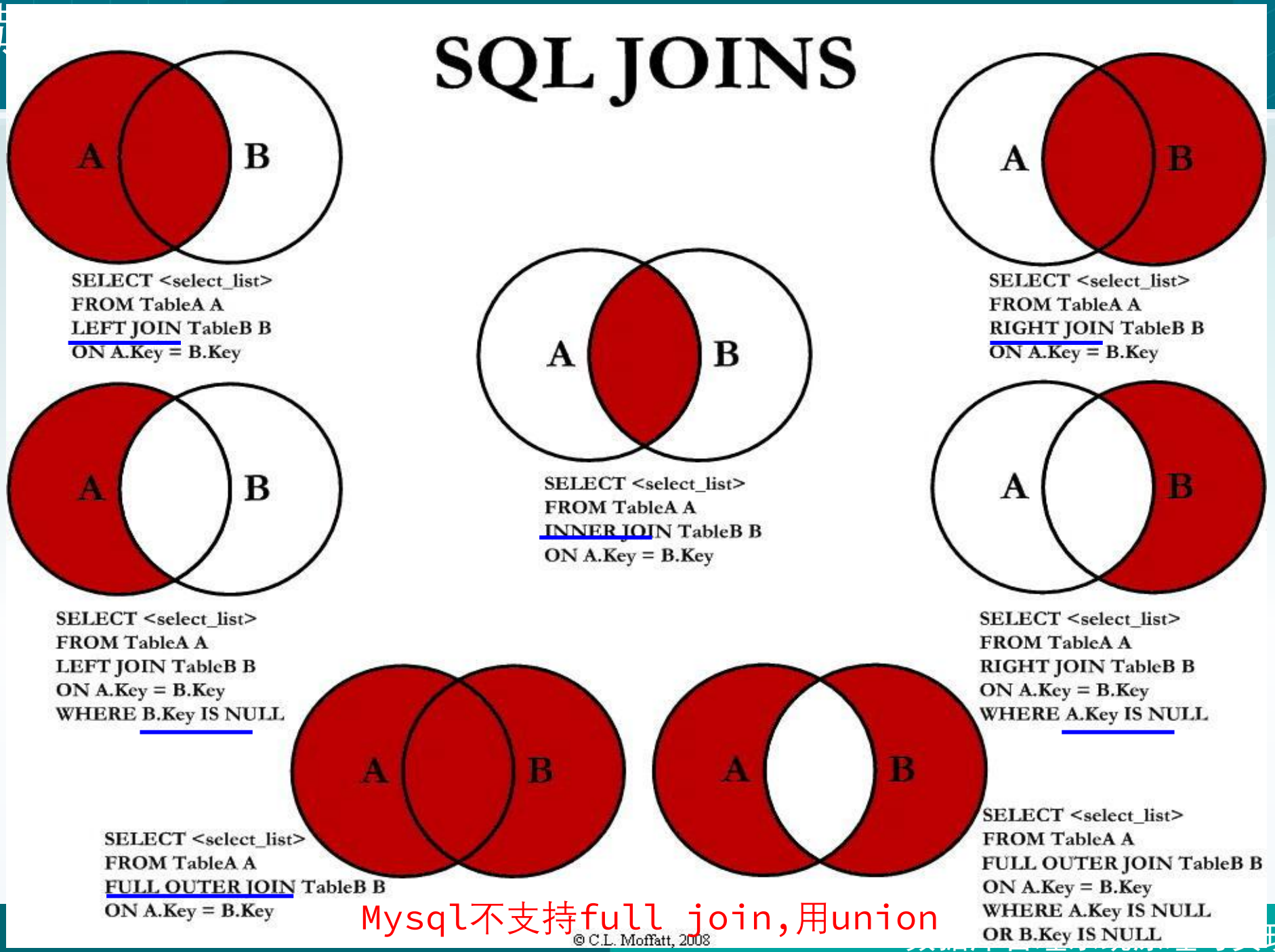


select * from a right join b on a.key = b.key

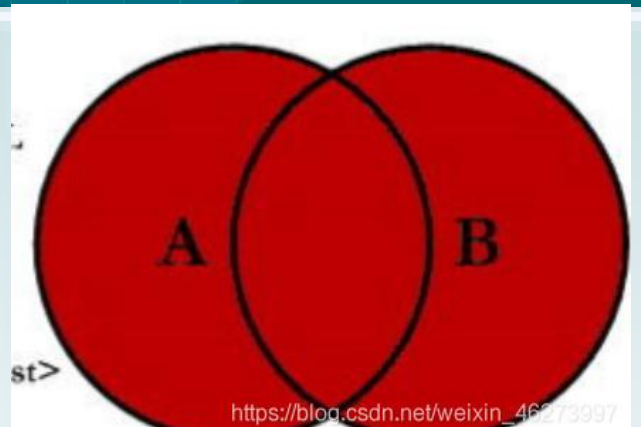



```
select * from a [inner] join b on a.key = b.key
```

集合查询（续

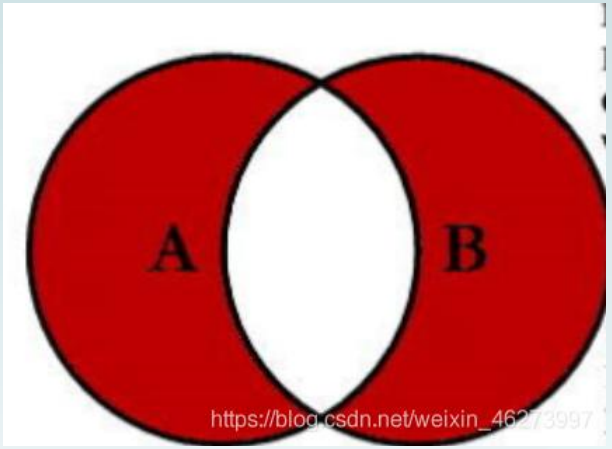

SQL JOINS
A
B
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key = B.Key
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
WHERE B.Key IS NULL
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
OR B.Key IS NULL
Mysql不支持full join,用union
© C.L. Moffatt, 2008

较少使用的外连接（续）




```
select * from a left
join b on a.key =
b.key
 union
select * from a right
join b on a.key =
b.key
```

SELECT column_name(s) FROM a
UNION SELECT column_name(s)
FROM b




```
select * from a left
join b on a.key = b.key
where b.key is null
union
select * from a right
join b on a.key = b.key
where a.key is null
```

连接查询（续）

1.等值与非等值连接查询

2.自身连接

3.外连接

4.多表连接

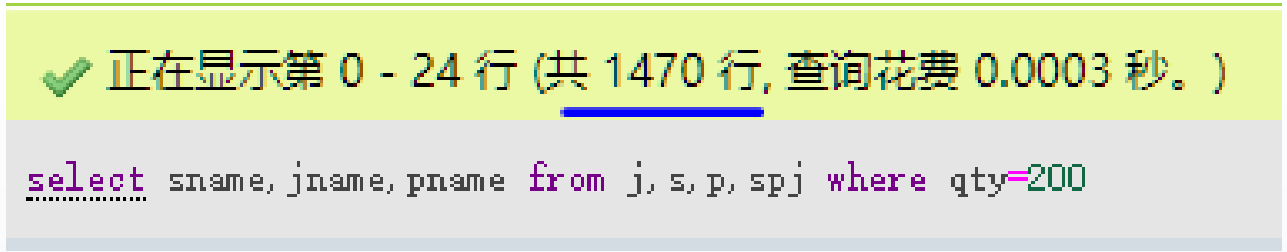
多表连接

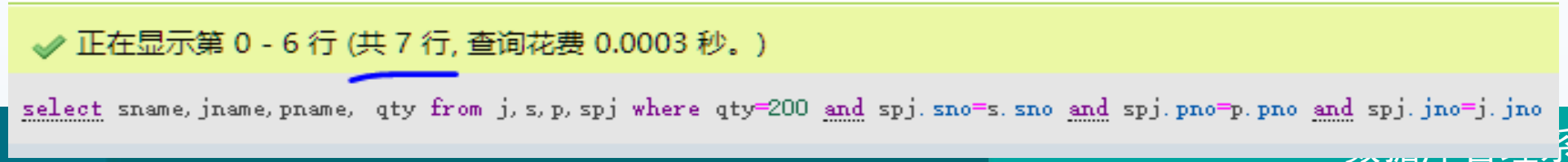
- 多表连接：两个以上的表进行连接

[例,第6版]查询每个学生的学号、姓名、选修的课程名及成绩

```
SELECT Student.Sno, Sname, Cname, Grade
FROM   Student, SC, Course    /*多表连接*/
WHERE Student.Sno = SC.Sno
      AND SC.Cno = Course.Cno;
```

#必须有等值连接条件，否则将出现笛卡尔积





3.3.2 连接查询

SQL标准允许从单个表或多个表中查询，若用到多个表，就涉及连接查询。

```
<joined table> ::=
<cross join>
| <qualified join>
| <left paren> <joined table> <right paren>
```

连接有两种类型，笛卡尔积（<cross join>）和限定连接（<qualified join>）。

3.3.2 连接查询

下面是笛卡尔积中的两个表用CROSS JOIN连接。

```
<cross join> ::=

<table reference> CROSS JOIN <table reference>
```

3.3.2 连接查询

[例3.12]用SQL描述Customers和Suppliers的笛卡尔积。

```
SELECT * FROM Products CROSS JOIN Suppliers;
```

查询结果：

CID	CName	Customers.City	SID	SName	Suppliers.City
1001	张三	北京	S001	华北电子厂	北京
1002	李四	广州	S001	华北电子厂	北京
1003	王五	上海	S001	华北电子厂	北京
1001	张三	北京	S002	西南印刷厂	成都
1002	李四	广州	S002	西南印刷厂	成都
1003	王五	上海	S002	西南印刷厂	成都

笛卡尔积的结果一般都无意义(比如几个大表)，
、因此除非很特殊的场景，否则均应避免使用

3.3.2 连接查询

用得最多的是限定连接，以下是个限定连接例子，查询供应商供应的商品信息：

```
SELECT Suppliers.*, PID
        FROM Suppliers
        LEFT JOIN Products ON Suppliers.SID= Products.SID;
```

教材注明：
Mysql省略OUTER

限定连接的一般语句为

```
<qualified join> ::=
<table reference> [ NATURAL ]
[ <join type> ] JOIN <table reference> [ <join specification> ]
```

3.3.2 连接查询

限定连接的连接类型（<join type>）可以是内连接（INNER）、外连接（OUTER），又分左连接（LEFT）和右连接（RIGHT）以及并（UNION）。

```
<join type> ::=
INNER | <outer join type> [ OUTER ] | UNION
```

3.3.2 连接查询

<join specification>描述两个表在哪个属性对上进行连接。

```
<join specification> ::=
<join condition> | <named columns join>
```

```
<join condition> ::= ON <search condition>
<named columns join> ::=
USING <left paren> <join column list> <right paren>
```

连接条件 ON 通常基于两个表中的列之间的匹配条件。
列名连接 USING 后面跟着括号，括号内列举了用于连接的列名。两个表中具有相同名称的列，它们的值相等时表示行匹配。

3.3.2 连接查询

[例3.13]查询“张三”创建的所有订单，显示CID，CName, OID，以及CreateTime。

```
SELECT Customers.CID,CName,OID,CreateTime
        FROM Customers  JOIN Orders ON Customers.CID= Orders.CID
        WHERE CName = '张三';
```

P45:INNER可省略

查询结果：

CID	CName	OID	CreateTime
1001	张三	O001	2023-1-1 18:00:10

3.3.2 连接查询

[例3.14]查询供应商所供应的商品信息，显示供应商的全部信息以及其供应商品的ID，要求将没有供应商品的供应商也显示出来 。

```
SELECT Suppliers.*, PID
        FROM Suppliers
        LEFT JOIN Products ON Suppliers.SID= Products.SID;
```

因为要显示没有供应商品的供应商，故采用了左外连接。

P45：mysql省略OUTER

查询结果：

SID	SName	City	PID
S001	华北电子厂	北京	P0001
S001	华北电子厂	北京	P0002
S001	华北电子厂	北京	P0003
S002	西南印刷厂	成都	P0004
S002	西南印刷厂	成都	P0005
S003	未央印刷厂	长安	NULL